SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:		**Reg. No.:**	
Date:		**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
2. **Project Structure Design**
   - Design and explain the folder structure of your project repository.
   - Justify the purpose of each major folder and file included in the repository.
3. **Git Repository Initialization**
   - Create a Git repository for the project.
   - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy**
   - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
   - Justify why the selected branching model is appropriate for the assigned project.
5. **Version Control Workflow**
   - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
   - Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
    - Present the commit history of the project.
    - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
    - Explain why Docker is suitable for the assigned project.
    - Identify the application components that require containerization.
8. **Dockerfile Development**
    - Design and explain the Dockerfile required to containerize the application.
    - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
    - Develop a Docker Compose configuration for the project.
    - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
    - Demonstrate the execution of the application using Docker containers.
    - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
    - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
    - Identify the challenges encountered during implementation.
    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository &	Repository initialized	Repository and	Basic Git setup	Incorrect or incomplete Git

Branching Strategy(20%)	correctly with appropriate branching strategy, clear workflow, and proper repository organization.	branching strategy are mostly correct.	with limited branching implementation.	repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

Intelligent AI-Based Emergency Shelter and Relief Camp Management Platform

Praveen Balamurugan

191571026

CSA1017

Code Review & Repository Evaluation

CO - 3 Assessment Tool - 1

INTELLIGENT AI-BASED EMERGENCY SHELTER AND RELIEF CAMP MANAGEMENT PLATFORM

Git & Docker Technical Assignment

PROBLEM ANALYSIS

1. Introduction

Natural disasters and humanitarian emergencies can occur suddenly and can affect a large number of people within a short period of time. Floods, earthquakes, cyclones, landslides, storms, and other disasters may force people to leave their homes and move to temporary emergency shelters. During such situations, government departments, disaster management authorities, non-governmental organizations, and volunteers must work together to provide food, water, medical care, accommodation, security, and other essential services.

Managing an emergency shelter manually can become difficult when the number of displaced people increases. Administrators may have to maintain information using paper records, spreadsheets, telephone calls, and separate communication channels. This can result in inaccurate information, delayed decisions, overcrowding, and inefficient resource distribution.

The proposed **Intelligent AI-Based Emergency Shelter and Relief Camp Management Platform** is designed to provide a centralized digital solution for managing emergency shelters and relief camps. The system uses digital information management and AI-based analysis to support shelter administrators in monitoring occupancy, managing resources, tracking supplies, coordinating volunteers, and generating reports.

The project also provides a suitable environment for applying software engineering concepts such as Git-based version control and Docker-based containerization.

1.1 Industry Context

During a disaster, emergency shelters must operate efficiently because displaced people depend on these facilities for their basic needs. Shelter managers need accurate information about the number of people staying at each location, the available food and medical supplies, and the number of volunteers available.

Modern technologies such as artificial intelligence, GIS, IoT, cloud computing, and mobile technologies can support real-time coordination of shelter operations. A digital platform can collect information from different sources and display it in a centralized dashboard.

The proposed system focuses on improving the management process by reducing dependence on manual records and providing administrators with useful information for decision-making.

1.2 Problem Statement

Emergency shelters often experience overcrowding, uneven distribution of relief resources, and poor coordination because accurate information may not be available immediately. Manual management methods make it difficult to continuously monitor occupancy, supplies, volunteer activities, and emergency requirements.

The proposed platform addresses these problems by providing digital shelter management facilities. It can maintain information about shelter capacity, current occupancy, food supplies, medical supplies, volunteers, and resource requirements.

The AI component can analyze available information and help identify shelters that require greater attention. The system can therefore support more efficient resource allocation and improve disaster response.

1.3 Project Objectives

The main objectives of the project are:

1. To monitor shelter occupancy.
2. To digitally manage relief resources.
3. To optimize resource allocation using AI.
4. To track food and medical supplies.
5. To coordinate volunteers efficiently.
6. To generate shelter management reports.
7. To improve emergency response.
8. To enhance humanitarian service delivery.

1.4 Expected Outcomes

The expected outcomes include efficient shelter operations, better utilization of resources, faster disaster response, improved humanitarian assistance, enhanced volunteer coordination, increased transparency, and better emergency planning.

PROJECT STRUCTURE DESIGN

2. Project Structure

A well-organized repository is important for developing and maintaining the emergency shelter management platform. Since the application contains several functionalities, the project should be divided into logical folders.

A suitable repository structure is shown below:

```
emergency-shelter-management/
│
├── frontend/
│   ├── components/
│   ├── pages/
│   ├── services/
│   ├── assets/
│   └── styles/
│
├── backend/
│   ├── controllers/
│   ├── models/
│   ├── routes/
│   ├── services/
│   └── middleware/
│
├── ai/
│   ├── models/
│   ├── training/
│   ├── preprocessing/
│   └── prediction/
│
├── database/
│   ├── schema/
│   └── seed/
```

```
│
├── tests/
│   ├── unit/
│   ├── integration/
│   └── system/
│
├── docs/
│   ├── diagrams/
│   └── screenshots/
│
├── Dockerfile
├── docker-compose.yml
├── .dockerignore
├── .gitignore
├── README.md
└── package.json
```

2.1 Frontend Folder

The `frontend` folder contains all user-interface-related code. It can contain pages for login, dashboard, shelter management, inventory, volunteers, reports, and alerts.

The `components` folder contains reusable interface components. The `pages` folder contains individual application screens. The `services` folder can contain API communication logic, while the `styles` folder contains styling files.

Separating frontend files makes the user interface easier to maintain and update.

2.2 Backend Folder

The backend contains the server-side application logic. It receives requests from the frontend, processes them, validates information, communicates with the database, and returns responses.

The `controllers` folder contains request-handling logic. The `routes` folder defines application endpoints. The `models` folder represents data structures, while the `services` folder contains reusable business logic.

2.3 AI Folder

The AI folder contains the components responsible for data preprocessing, model training, prediction, and resource allocation analysis.

Keeping AI functionality separate from the main application improves maintainability and allows the model to be updated without changing unrelated parts of the application.

2.4 Database Folder

The database folder contains database schema definitions and initial data. It can store information related to shelters, users, inventory, volunteers, occupancy, and reports.

2.5 Tests Folder

The tests folder contains unit tests, integration tests, and system-level tests. Separating test files from application code makes the project easier to understand.

2.6 Documentation Folder

The documentation folder can contain architecture diagrams, workflow diagrams, screenshots, and other supporting documents.

This organization supports the assignment requirement to provide diagrams, screenshots, commands, and explanations.

GIT REPOSITORY INITIALIZATION

3. Git Repository Initialization

Git is a distributed version control system that allows developers to track changes in project files. Git is particularly useful for software projects because it maintains a history of changes and allows developers to work on different features without affecting the stable version of the application.

The emergency shelter platform can be maintained using a Git repository hosted on GitHub or another Git-based platform.

3.1 Initializing the Repository

The repository can be initialized using the following commands:

mkdir emergency-shelter-management

cd emergency-shelter-management

git init

The `git init` command creates a new Git repository in the project directory.

The project files can then be added using:

git add .

After adding the files, the first commit can be created:

git commit -m "Initial project setup"

A remote repository can then be connected:

git remote add origin <repository-url>

The project can be uploaded using:

git push -u origin main

3.2 Purpose of Git Files

.git Directory

The `.git` directory contains Git's internal information. It stores commit history, branch information, configuration, and other repository metadata.

This directory should not normally be modified manually.

.gitignore

The `.gitignore` file specifies files that should not be committed to the repository.

For example:

node_modules/

.env

dist/

build/

*.log

This prevents unnecessary files and sensitive environment information from being stored in the repository.

README.md

The README file provides information about the project. It should explain the project purpose, features, installation process, technologies, usage instructions, and repository structure.

package.json

For a Node.js-based application, `package.json` contains project metadata and dependencies.

3.3 Importance of Git

Git provides a history of project development. If a developer accidentally introduces an error, an earlier version can be reviewed or restored.

Git also makes collaboration easier because multiple developers can work on different branches and merge their completed changes.

Suggested screenshot:
[Insert Screenshot – `git init` command]

Suggested screenshot:
[Insert Screenshot – GitHub repository]

GIT BRANCHING STRATEGY

4. Git Branching Strategy

A branching strategy defines how developers organize different versions of source code. For the proposed emergency shelter platform, a structured branching model can be used.

The following branches are suitable:

main

```
  │
  └── develop
      │
      ├── feature/shelter-management
      ├── feature/inventory
      ├── feature/volunteer-management
      ├── feature/ai-resource-allocation
      └── feature/dashboard
```

4.1 Main Branch

The `main` branch contains stable versions of the application. Code should be merged into this branch only after appropriate testing.

The main branch represents the version that can be considered ready for deployment.

4.2 Develop Branch

The `develop` branch contains features that have been implemented but may still require additional testing before release.

Developers can merge completed feature branches into the development branch.

4.3 Feature Branches

Each major feature can be developed using a separate branch.

For example:

git checkout -b feature/shelter-management

After completing the shelter management functionality, the developer can commit the changes:

git add .

git commit -m "Add shelter management module"

The feature branch can then be pushed:

git push origin feature/shelter-management

4.4 Release Branch

A release branch can be created when the application reaches a stage where it is ready for final testing.

Example:

release/v1.0

This branch can be used for final bug fixes and release preparation.

4.5 Hotfix Branch

If a critical problem occurs in the production version, a hotfix branch can be created.

For example:

hotfix/inventory-validation

After the issue is corrected and tested, the hotfix can be merged into the main branch.

4.6 Why This Strategy Is Suitable

This branching strategy is suitable because the project contains multiple independent modules. Developers can work on shelter management, inventory management, AI functionality, and dashboards without directly modifying the stable version.

The assignment specifically asks students to design a suitable branching strategy and justify why the selected model is appropriate.

VERSION CONTROL WORKFLOW AND COMMIT HISTORY

5. Version Control Workflow

A proper Git workflow ensures that project changes are recorded systematically.

The general workflow can be represented as:

Working Directory

↓

git add

↓

Staging Area

↓

git commit

↓

Local Repository

↓

git push

↓

Remote Repository

5.1 Creating a Feature

A developer begins by creating a feature branch:

git checkout -b feature/inventory

The developer then implements the required functionality.

After completing the changes:

git status

The `git status` command shows modified and untracked files.

The changes can be staged:

git add .

The changes can then be committed:

git commit -m "Add relief inventory management"

Finally, the changes can be pushed:

git push origin feature/inventory

5.2 Merging

After testing, the feature can be merged into the development branch.

git checkout develop

git merge feature/inventory

If there are no conflicts, Git completes the merge automatically.

5.3 Conflict Resolution

A merge conflict can occur when two branches modify the same section of a file differently.

The developer must inspect the conflicting file, select the correct version, remove conflict markers, and then commit the resolved changes.

Example:

git add .

git commit -m "Resolve merge conflict"

Conflict resolution is an important part of collaborative software development because different developers may work on related parts of the project.

5.4 Repository Synchronization

Before starting new work, developers should obtain the latest version of the project:

git pull origin develop

This reduces the possibility of working with outdated code.

5.5 Meaningful Commit Messages

Commit messages should clearly explain what changed.

Good examples include:

Add shelter occupancy module

Implement volunteer registration

Add medical inventory tracking

Create AI resource allocation service

Fix shelter capacity validation

Update Docker configuration

Add project documentation

Poor commit messages include:

update

changes

final

new

test

Meaningful messages make the repository history easier to understand and maintain.

Suggested screenshot:
[Insert Screenshot – Commit history]

Suggested screenshot:
[Insert Screenshot – Branch creation]

DOCKER ENVIRONMENT DESIGN

6. Docker Environment Design

Docker is a containerization platform that allows applications and their dependencies to be packaged into isolated environments called containers.

Docker is suitable for the emergency shelter management platform because the project may contain multiple components with different dependencies.

For example, the application may contain:

1. Frontend service.
2. Backend service.
3. Database service.
4. AI service.

Instead of installing every dependency manually on each computer, Docker can provide a consistent environment.

6.1 Why Docker Is Suitable

One major advantage of Docker is portability. A container can run on different systems as long as Docker is available.

This reduces the common problem of an application working on one developer's computer but failing on another developer's computer because of different software versions.

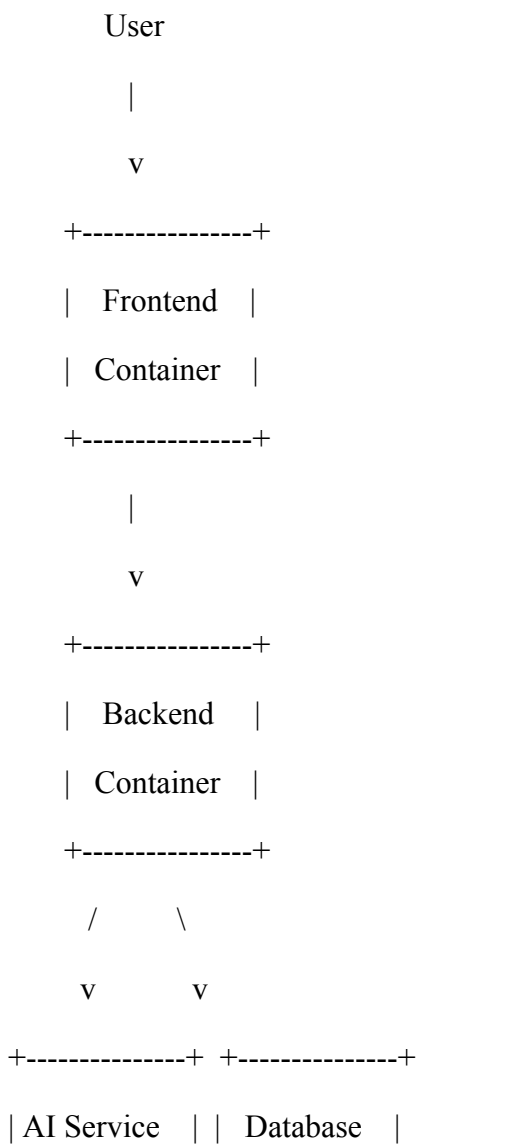
Docker also improves deployment consistency. The same container configuration used during development can be used during testing and deployment.

6.2 Components Requiring Containerization

The frontend can be placed in one container. The backend can run in another container. The database can run as a separate service.

If the AI component requires a different environment or programming language, it can also be containerized separately.

A possible architecture is:



| Container | | Container |

+---------------+ +---------------+

6.3 Docker Benefits

Docker provides portability, isolation, reproducibility, easier deployment, dependency management, and scalability.

It also makes it easier for team members to create similar development environments.

DOCKERFILE DEVELOPMENT

7. Dockerfile Development

A Dockerfile contains instructions that Docker uses to create an application image.

For a Node.js-based backend, a basic Dockerfile can be designed as follows:

```
FROM node:20


WORKDIR /app


COPY package*.json ./


RUN npm install


COPY . .


EXPOSE 5000
```

CMD ["npm", "start"]

7.1 FROM Instruction

FROM node:20

The `FROM` instruction specifies the base image. In this example, the Node.js 20 image provides the runtime environment required by the application.

7.2 WORKDIR Instruction

WORKDIR /app

The `WORKDIR` instruction specifies the working directory inside the container.

All subsequent commands are executed relative to this directory.

7.3 COPY Instruction

COPY package*.json ./

This copies the package configuration files into the container.

The dependencies can then be installed.

7.4 RUN Instruction

RUN npm install

This installs the dependencies required by the application.

7.5 COPY Application Files

COPY . .

This copies the remaining project files into the container.

7.6 EXPOSE Instruction

EXPOSE 5000

This indicates the port on which the application is expected to run inside the container.

7.7 CMD Instruction

CMD ["npm", "start"]

This specifies the default command that runs when the container starts.

7.8 Building the Docker Image

The image can be built using:

docker build -t emergency-shelter-app .

The `-t` option gives the image a meaningful name.

7.9 Running the Container

The container can be started using:

docker run -p 5000:5000 emergency-shelter-app

The port mapping allows users to access the application from the host machine.

Suggested screenshot:
[Insert Screenshot – Docker image build]

DOCKER COMPOSE DESIGN

8. Docker Compose Design

Docker Compose is useful when an application contains multiple containers that need to communicate with each other.

The emergency shelter platform may contain frontend, backend, AI, and database services. Docker Compose can define all these services in a single configuration file.

A sample `docker-compose.yml` is:

```
services:


  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      - DATABASE_URL=mongodb://database:27017/emergency_shelter
    depends_on:
      - database


  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend
```

```
  database:
    image: mongo:7
    ports:
      - "27017:27017"
    volumes:
      - shelter_data:/data/db


volumes:
  shelter_data:
```

8.1 Backend Service

The backend service contains the server-side application. It communicates with the database and provides APIs to the frontend.

The backend exposes port 5000.

8.2 Frontend Service

The frontend service contains the user interface. Users can access the application through port 3000.

The frontend depends on the backend because it requires backend APIs to obtain and update information.

8.3 Database Service

The database service stores application information.

A persistent volume is used so that database information is not lost when the container is stopped or recreated.

8.4 Networking

Docker Compose automatically provides a network for the services. Services can communicate using their service names.

For example:

backend → database

The backend can connect to the database using the service name `database`.

8.5 Environment Variables

Environment variables can be used to configure database URLs, API keys, ports, and other environment-specific values.

Sensitive information should not be directly committed to the Git repository.

8.6 Starting the Application

The complete environment can be started using:

docker compose up --build

The `--build` option ensures that the required images are built before starting the services.

To stop the services:

docker compose down

Suggested screenshot:
[Insert Screenshot – Docker Compose execution]

PAGE 9 – CONTAINER DEPLOYMENT AND TESTING

9. Container Deployment and Testing

After creating the Dockerfile and Docker Compose configuration, the application should be tested inside containers.

The purpose of container testing is to verify that the application works correctly in the configured environment.

9.1 Starting the Containers

The application can be started using:

docker compose up --build

Docker builds the required images and starts the defined services.

The running containers can be checked using:

docker ps

This command displays the currently running containers.

9.2 Testing the Frontend

The frontend can be opened in a web browser using:

http://localhost:3000

The dashboard should load successfully.

The administrator should be able to navigate to the shelter management, inventory, volunteer, and reporting sections.

9.3 Testing the Backend

The backend should be tested by sending requests to the appropriate API endpoints.

For example:

GET /api/shelters

The expected result is a valid response containing shelter information.

9.4 Shelter Occupancy Test

A shelter can be created with a predefined capacity.

Example:

Shelter Capacity = 500

Current Occupancy = 350

The application should display the correct occupancy information.

If occupancy approaches the maximum capacity, the dashboard can display an appropriate warning.

9.5 Inventory Test

The administrator can add food and medical supplies.

Example:

Food Packets = 1000

Medical Kits = 200

Water Bottles = 1500

The system should display these quantities correctly.

When resources are distributed, the available quantity should be updated.

9.6 Volunteer Test

The administrator can register a volunteer and assign the volunteer to a shelter.

The system should maintain the volunteer's details and assignment.

9.7 AI Resource Allocation Test

The AI module can evaluate shelter conditions based on occupancy and resource availability.

For example:

Shelter A:

Occupancy = 95%

Food Stock = Low

Medical Stock = Low

Shelter B:

Occupancy = 50%

Food Stock = High

Medical Stock = Medium

The system should identify Shelter A as requiring higher priority.

9.8 Container Health Verification

The administrator can inspect container logs using:

docker compose logs

If a service fails, the logs can help identify the cause.

9.9 Testing Results

Testing should verify that:

- Containers start successfully.
- Frontend loads correctly.
- Backend responds correctly.
- Database connection works.
- Shelter data can be stored.
- Occupancy information can be updated.
- Inventory information can be updated.
- Volunteers can be managed.
- AI recommendations can be generated.
- The application remains functional after restarting containers.

The assignment specifically expects evidence that the application runs successfully, including screenshots of Docker execution and testing.

Suggested screenshots:

- Docker containers running.
- Application running in browser.
- Dashboard.
- Shelter management.

- Inventory.
- Docker logs.
- Test results.

BENEFITS, CHALLENGES AND FUTURE ENHANCEMENTS

10. Benefits of Git and Docker

Git and Docker provide significant benefits to the emergency shelter management project.

10.1 Collaboration

Git allows multiple developers to work on different features simultaneously. Feature branches make it possible for developers to work independently and merge their completed work later.

This is particularly useful when one developer works on the frontend while another works on backend services and another works on AI functionality.

10.2 Version Management

Git maintains the history of changes. Developers can identify when a change was introduced and who made the change.

If an error occurs, previous versions can be reviewed to identify the source of the problem.

10.3 Portability

Docker packages the application and its dependencies into containers. This allows the project to run consistently on different development machines.

10.4 Deployment

Docker simplifies deployment because the application environment can be recreated using the Dockerfile and Docker Compose configuration.

10.5 Scalability

Containerized services can be scaled independently when required. For example, if backend traffic increases, additional backend instances can be created without necessarily changing the frontend.

10.6 Maintainability

Git provides organized version history, while Docker provides a consistent application environment. Together, they improve maintainability.

---

11. Challenges

Several challenges may occur while implementing the project.

11.1 Git Merge Conflicts

Merge conflicts can occur when multiple developers modify the same files. Resolving these conflicts requires developers to understand the differences between the branches.

11.2 Dependency Problems

Different versions of programming languages, libraries, or databases can cause compatibility problems.

Docker reduces this problem by providing a consistent environment.

11.3 Docker Configuration

Creating a Dockerfile requires understanding application dependencies, ports, working directories, and startup commands.

Incorrect configuration can prevent the application from starting correctly.

11.4 Database Connectivity

The backend must connect correctly to the database container. Incorrect connection strings, ports, or environment variables can cause connection failures.

11.5 Resource Usage

Running multiple containers may consume considerable system resources. During development, unnecessary containers should be stopped when they are not required.

11.6 AI Model Integration

The AI component may require additional libraries, datasets, and computational resources. Integrating the model with the main application may therefore require careful API and environment design.

---

12. Future Enhancements

The system can be enhanced in several ways.

12.1 IoT Integration

IoT sensors can be used to automatically collect occupancy information from shelters.

This can reduce manual data entry and provide more frequently updated information.

12.2 GIS Integration

GIS technology can be integrated to display emergency shelters on maps. Administrators can view shelter locations, capacity, occupancy, and available resources geographically.

12.3 Mobile Application

A mobile application can allow volunteers and shelter managers to update information from smartphones.

This would be useful when administrators are working away from desktop computers.

12.4 AI-Based Demand Prediction

Future versions can use historical disaster data, shelter occupancy, weather information, and resource consumption to predict future demand.

The system could then provide early recommendations for resource preparation.

12.5 Automated Alerts

The platform can generate alerts when:

- Shelter occupancy becomes critical.
- Food supplies become low.
- Medical supplies reach minimum levels.
- A shelter becomes unavailable.
- Additional volunteers are required.

12.6 Cloud Deployment

The application can be deployed on cloud infrastructure to improve availability and scalability.

Cloud deployment can allow multiple government departments and relief organizations to access centralized information when appropriate security controls are implemented.

12.7 CI/CD Integration

Git can be integrated with a continuous integration and continuous deployment pipeline. Whenever changes are pushed to the repository, automated tests can be executed before a new Docker image is deployed.

This would create a more reliable development and deployment process.

---

13. CONCLUSION

The Intelligent AI-Based Emergency Shelter and Relief Camp Management Platform provides a centralized approach for managing emergency shelters and humanitarian relief operations.

The platform addresses important challenges such as shelter overcrowding, resource tracking, volunteer coordination, and emergency information management. The use of AI can provide additional decision support for allocating resources according to demand.

Git provides an organized version control system for the project. Through branches, meaningful commits, merging, synchronization, and repository history, developers can collaborate efficiently while maintaining a reliable development history.

Docker provides a consistent environment for running the application. The Dockerfile defines the application environment, while Docker Compose can coordinate multiple services such as the frontend, backend, and database.

The combination of Git and Docker improves collaboration, portability, deployment, scalability, and maintainability. It also supports the software engineering goal of building full-stack applications with an emphasis on containerization, version control, and collaborative development, which is the stated CO3 focus of the assignment.

The project can be further improved through IoT-based occupancy monitoring, GIS integration, mobile applications, predictive AI, automated alerts, cloud deployment, and CI/CD pipelines.

Overall, the proposed system demonstrates how modern software engineering practices can be applied to a humanitarian problem. A well-organized Git repository combined with a properly designed Docker environment provides a strong foundation for developing, testing, and deploying the emergency shelter management platform.

REFERENCES / SUPPORTING EVIDENCE

Screenshots to Include

1. Git repository creation.
2. `git init` command.
3. GitHub repository structure.
4. `.gitignore` file.
5. Git branches.
6. Git commit history.
7. Git merge operation.
8. Dockerfile.
9. Docker image build.
10. Docker Compose configuration.
11. Running Docker containers.
12. Application dashboard.
13. Shelter occupancy screen.
14. Resource inventory screen.
15. Volunteer management screen.
16. AI resource allocation screen.
17. Docker logs.
18. Successful application execution.

Diagrams to Include

Diagram 1 – Project Architecture

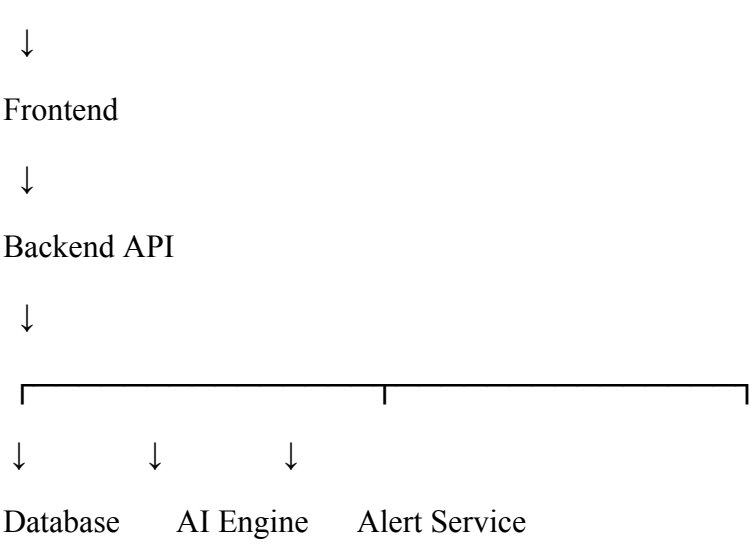


Diagram 2 – Git Workflow

Feature Branch

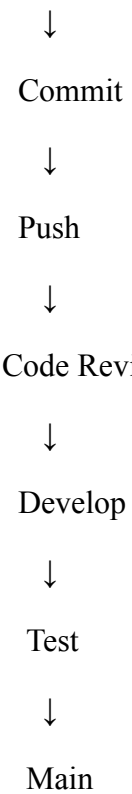


Diagram 3 – Docker Architecture

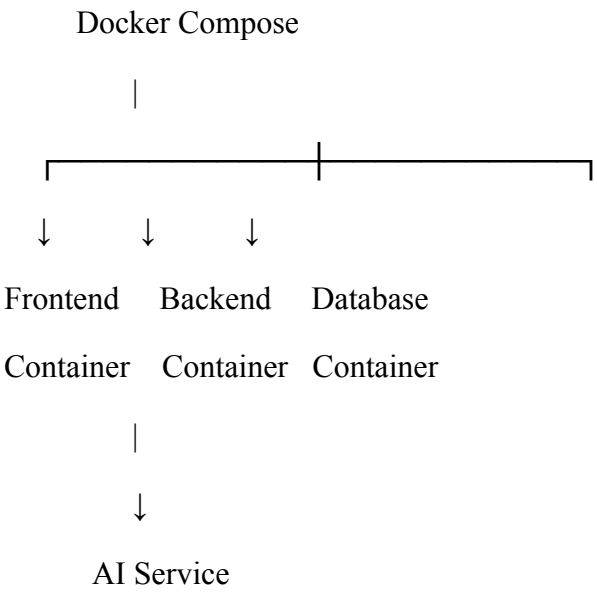


Diagram 4 – Application Workflow

User Login

↓

Dashboard

↓

Shelter Monitoring

↓

Resource Tracking

↓

AI Analysis

↓

Resource Recommendation

↓

Volunteer Coordination

↓

Report Generation

Final Submission Checklist

- 10-page technical report
- Git repository
- Complete commit history
- Source code
- Dockerfile
- docker-compose.yml
- Git operation screenshots
- Docker execution screenshots
- Architecture diagram
- Git workflow diagram
- Docker architecture diagram
- Testing evidence
- Challenges
- Future enhancements